



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)  
'walls': list(self.walls)  
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **state space** is the complete set of all possible states that can exist in the environment. In this grid game, each valid coordinate that the agent can occupy is a state. The state space therefore represents all possible positions of the agent, excluding invalid positions such as walls.

The **search tree** is the structure created by a search algorithm while exploring the state space. Each node in the search tree represents a state, while the branches represent actions such as moving UP, DOWN, LEFT, or RIGHT.

The main difference is that the state space describes all possible states in the problem, while the search tree represents the particular paths explored by the search algorithm. The same state may appear through different paths in a search tree unless repeated states are prevented using graph-search techniques.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set is used to keep track of states that have already been discovered or visited by the search algorithm.

Its main purpose is to prevent the agent from repeatedly visiting the same states and creating cycles. For example, without a reached set, the agent could move from one cell to another and then return to the previous cell repeatedly.

This converts the algorithm from a tree search into a graph search.

The reached set is especially important for DFS. Without it, DFS could repeatedly follow cycles in the grid and continue exploring the same positions indefinitely. This could cause the search to become extremely slow or even result in an infinite loop.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces

winding, suboptimal routes?

BFS explores the search space level by level. It first explores all states that are one move away from the starting position, then states that are two moves away, then three moves away, and so on.

Because each movement in this grid has the same cost, the first time BFS reaches the goal, it has found a path containing the minimum possible number of moves. Therefore, BFS is guaranteed to produce an optimal path in this environment.

DFS behaves differently. It follows one branch as deeply as possible before backtracking. Because of this, it may travel through a long sequence of cells before eventually finding the food.

This explains why DFS can produce winding or inefficient routes even when a much shorter route exists nearby. DFS is not guaranteed to find the shortest path.

Therefore, in this grid:

BFS produces a shortest path because all actions have equal cost.

DFS finds a valid path but it may be much longer and less direct.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

For a very large grid, memory usage becomes an important problem.

The typical time and space complexity of BFS is:

Time: $O(b^d)$

Space: $O(b^d)$

where:

b is the branching factor

d is the depth of the shallowest goal

BFS stores many nodes at the same time because it must keep all nodes on the current search level in the frontier. As the search depth increases, the number of

stored nodes can grow very quickly. Therefore, BFS has a high memory requirement and could run out of memory on a very large grid.

DFS has approximately:

Time: $O(b^m)$

Space: $O(bm)$

where:

b is the branching factor

m is the maximum depth of the search space

DFS mainly stores the current search path together with a smaller number of unexplored alternatives. Therefore, its memory requirement is much lower than BFS.

On a 1000×1000 grid, BFS could consume a large amount of memory because its frontier may contain a very large number of coordinates. DFS would normally require much less memory, although it may explore extremely deep and inefficient paths before finding the goal.

Therefore, BFS is generally more memory-intensive, while DFS is more memory-efficient but does not guarantee an optimal solution.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING